

How Attention Works — and the KV Cache

The mechanism behind every transformer, and the optimization that makes chat feel instant

Client: Concept Explainer

Trainer: Advanced Agentic AI Bootcamp

Date: Foundations



THE PUZZLE

Why the First Word Is Slow — Then Fast

Notice it in ChatGPT or Claude: the first token takes a beat, then words stream steadily. That's not the network — it's how transformers generate text, and the KV cache is why it's usable at all.



3 Seconds vs 45 Seconds

The same answer, with and without the KV cache: **roughly 3 seconds vs 45 seconds**. The KV cache is what makes conversational AI actually responsive.

To understand the cache, you first have to see what the model is doing on every single token:

- A transformer generates one token at a time — autoregressively.
- To produce the next token, it runs the attention mechanism over every token so far.
- Naively, that means re-doing work for the whole sequence on every new token — slower and slower as the text grows.
- The KV cache removes that repeated work. First, let's see how attention itself works.



PART 1

How Attention Works

Every transformer uses attention to weigh how important each word is when interpreting the text. It works through three components: Queries, Keys, and Values.



Q, K, V — Think of a Database Lookup

Query (Q)

**What you're
searching for**

The current token asking: 'what's relevant to me?'

Key (K)

**The indices you
search through**

Every token's label — matched against the query

Value (V)

**The actual data
returned**

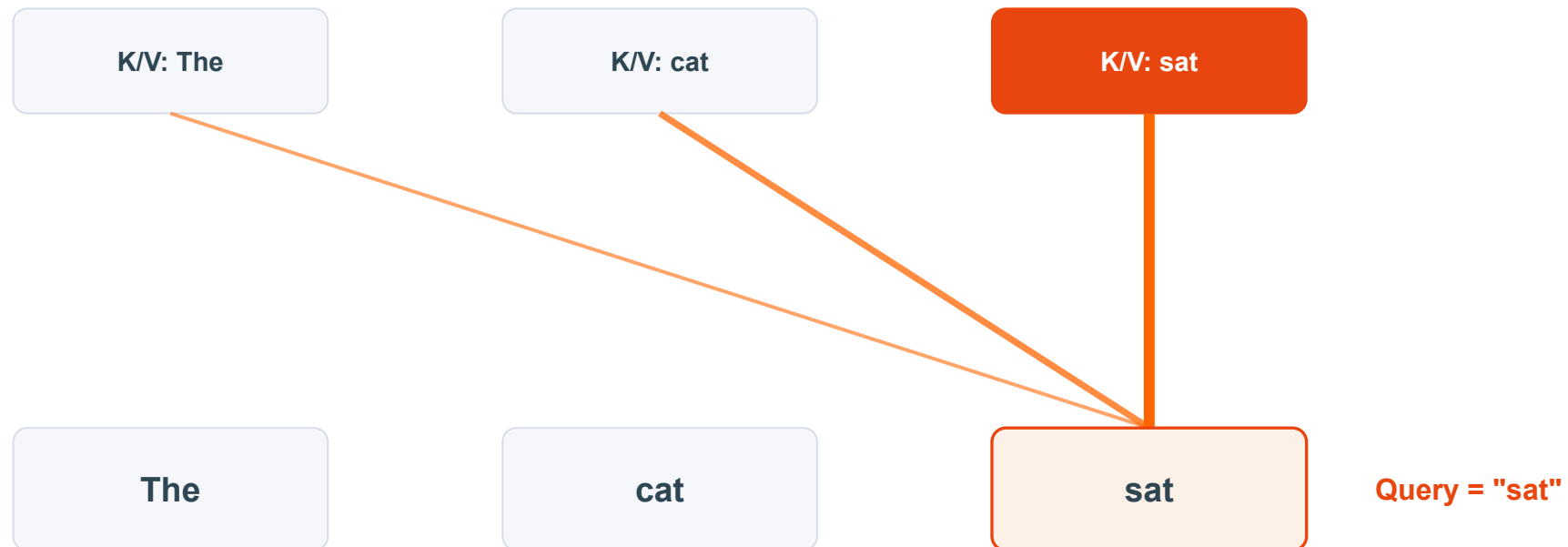
The content pulled from the tokens that matched

The model computes how well each Query matches every Key, turns those scores into weights, and blends the Values accordingly. High match = that token gets more say in the output.



Attention in a Picture

The token "sat" looks at every earlier token — line thickness = how much attention it pays to each:



Thick line = strong attention. "sat" attends most to "cat" (the subject) and to itself — that weighting is how the model decides what each word means in context.



The Key Insight for What Follows

For every token the model processes, it computes a Key and a Value. Those K and V vectors describe that token — and crucially, they don't change when later tokens arrive.

- When generating token #100, the model needs the K and V of tokens #1–#99 to run attention.
- Those were already computed when tokens #1–#99 were first processed.
- So why recompute them every step? That waste is exactly what the KV cache eliminates.



PART 2

The KV Cache

A simple idea with a huge payoff: compute each token's Key and Value once, store them, and reuse them for every future token instead of recomputing.



Without vs With the Cache

WITHOUT a KV cache

- Every new token re-runs attention over the **WHOLE sequence**
- Re-computes K and V for every previous token, every step
- Work per token grows with sequence length
- Generation gets slower and slower as text grows
- → the 45-second answer

WITH a KV cache

- Each token's K and V are computed once and stored
- New token computes only its OWN K and V
- Then attends against the cached K/V of all prior tokens
- Work per token stays roughly constant
- → the 3-second answer



Prefill vs Decode — the Two Phases

Prefill (the first-word wait)

- The model processes your entire prompt at once
- Computes K and V for every prompt token → fills the cache
- This is the heavy, parallel compute step
- It's why the FIRST token takes a moment to arrive
- Long prompts = longer prefill = longer first-token wait

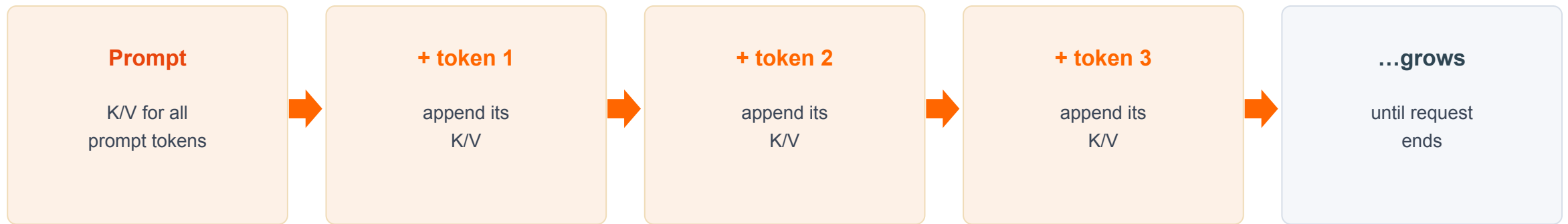
Decode (the steady stream)

- Now the model generates one token at a time
- Each step: compute K/V for just the new token, append to cache
- Attend against the whole cache — no recompute
- Fast and roughly constant per token
- This is why words stream out at a steady pace



The Cache Grows as It Generates

Each generated token adds its Key and Value to the cache — it accumulates for the whole request:



The trade-off — and the bridge to deployment

- The KV cache trades **MEMORY** for **SPEED** — you store K/V so you don't recompute it.
- That stored cache lives in GPU memory and grows with context length × number of concurrent users.
- At long context and real concurrency it can exceed the model weights themselves — which is why it drives GPU sizing (see the concurrency briefing).



The Whole Story in Five Lines

- 1 · Attention weighs how much each token should attend to every other token, via Queries, Keys, and Values.
- 2 · A token's Key and Value are computed once and never change when later tokens arrive.
- 3 · Without a cache, the model recomputes all K/V every step — work grows with length, generation crawls.
- 4 · The KV cache stores each token's K/V and reuses it — prefill fills the cache, decode appends to it and streams fast.
- 5 · It trades GPU memory for speed — and that memory cost is what every architect must size for.



Grounded Reference

"What the hell is a KV Cache?" — GenAI Nexus (Medium, 2025)

<https://medium.com/genai-nexus/what-is-a-kv-cache-f4c610c0f79d>

A deep dive into the memory-speed trade-off in LLM inference — the source for the Q/K/V analogy and the prefill/decode framing.

Go deeper

- The original transformer paper: "Attention Is All You Need" (Vaswani et al., 2017) — arxiv.org/abs/1706.03762
- For the deployment consequence — how the KV cache drives GPU concurrency limits — see the KV Cache Concurrency briefing.
- Optimizations that shrink the cache: PagedAttention (vLLM), FP8 KV cache, prefix caching.

Compute Once, Reuse Forever

That's the KV cache in one line — and the reason your GPU memory, not your model size, decides how many users you can serve.
